

SiliconJackets Digital Design Onboarding Write-up

Screenshot showing your submission passes the make verify_onboarding command

```
[akumar933@ece-rschsrv verilog]$ tcsh

Time:          0
-----Initializing Signals-----

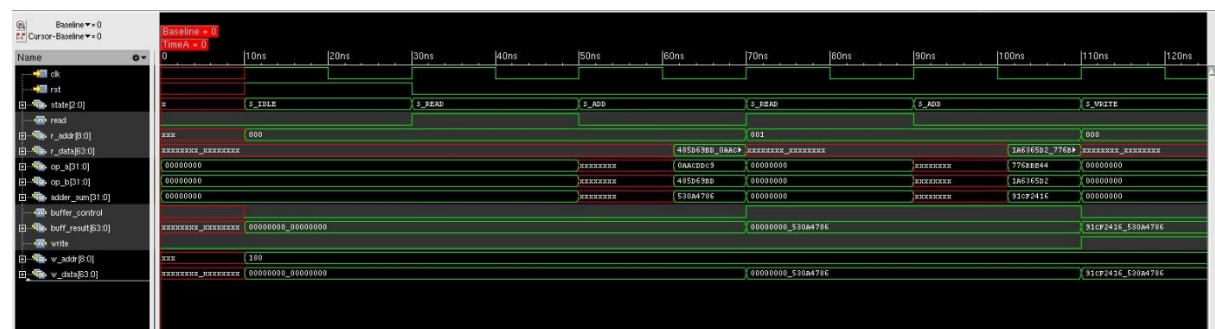
Time:          1000
-----Finished Simulation!-----

Time:          1293000
Simulation complete via $finish(1) at time 12930 NS + 0
./sym_links/tb_calculator.sv:52      $finish;
xcelium> exit

coverage setup:
workdir   : ./cov_work
dutinst   : tb_calculator(tb_calculator)
scope     : scope
testname  : test

coverage files:
model(design data) : ./cov_work/scope/icc_28d6a679_00000000.ucm
data              : ./cov_work/scope/test/icc_28d6a679_00000000.ucd
TOOL:  xrun(64)      25.03-s003: Exiting on Sep 14, 2025 at 13:55:09 EDT (total: 00:00:01)
make[1]: Leaving directory '/nethome/akumar933/Digital-Design-Onboarding/sim/behav'
python3.12 ../../scripts/check_onboarding.py /nethome/akumar933/Digital-Design-Onboarding/sim/behav/WORKSPACE/memory_post_state_lower.txt /nethome/akumar933/Digital-Design-Onboarding/sim/behav/WORKSPACE/sim_memory_post_state_lower.txt
PASSED: RTL simulation output matches expected results.
python3.12 ../../scripts/check_onboarding.py /nethome/akumar933/Digital-Design-Onboarding/sim/behav/WORKSPACE/memory_post_state_upper.txt /nethome/akumar933/Digital-Design-Onboarding/sim/behav/WORKSPACE/sim_memory_post_state_upper.txt
PASSED: RTL simulation output matches expected results.
akumar933@ece-rschsrv.ece.gatech.edu>
```

Screenshot showing the Simulation Waveform



FSM Design and Operation

The core of the calculator is a finite-state machine (FSM) defined in controller.sv. This FSM acts as the brain of the operation, directing the flow of data by generating the necessary control signals in a precise sequence. It cycles through several states to fetch operands from memory, perform calculations, and store the results.

Analysis of FSM States

The controller uses five distinct states to manage the calculation process, as shown by the state[2:0] signal in the waveform.

S_IDLE

Purpose: This is the initial state after a system reset.

Actions: The controller ensures all control signals like read and write are de-asserted (low). It initializes internal address pointers from the module's inputs.

Transition: The FSM stays in this state for only one clock cycle before unconditionally transitioning to S_READ to begin the first calculation. This is visible at the very beginning of the waveform.

S_READ

Purpose: To fetch a 64-bit word from the two parallel SRAMs.

Actions: The controller asserts the read signal to enable the SRAMs' read ports. Simultaneously, it outputs the correct memory address on the r_addr bus.

Transition: After one cycle, it always transitions to S_ADD to process the fetched data.

In the Waveform: You can see the FSM enters S_READ twice in a full cycle. Each time, the read signal goes high. Shortly after, the r_data bus updates with the value read from memory.

S_ADD

Purpose: To perform the 32-bit addition and buffer the result.

Actions: The controller takes the 64-bit word from the r_data input and splits it into two 32-bit operands: op_a (lower 32 bits) and op_b (upper 32 bits). These operands are sent to the external adder, which calculates the adder_sum. This sum is then stored in the result_buffer.

Transition: The next state depends on whether this is the first or second addition in the pair:

First Add: If buffer_control is 0, the sum is stored in the lower half of the 64-bit result_buffer, and the FSM transitions back to S_READ to fetch the second word.

Second Add: If buffer_control is 1, the sum is stored in the upper half of the buffer. Since the buffer is now full with both results, the FSM transitions to S_WRITE.

In the Waveform: The FSM enters S_ADD after each read. You can see op_a and op_b receive the values from r_data, and the adder_sum is computed. The buff_result signal shows how the first sum is latched, and then the second sum is combined with it in the next S_ADD state.

S_WRITE

Purpose: To write the combined 64-bit result from the buffer back to memory.

Actions: The controller asserts the write signal to enable the SRAMs' write ports. It places the destination address on w_addr and the full 64-bit result from buff_result onto the w_data bus.

Transition: After one cycle, it checks if the final address has been reached. If not, it transitions back to S_READ to start the next calculation. If the task is complete, it moves to S_END.

In the Waveform: At the end of the cycle, the state becomes S_WRITE. The write signal goes high, and the final combined value from buff_result appears on w_data to be stored.

S_END

Purpose: A terminal state to halt the calculator's operation.

Actions: No operations are performed. The controller remains idle.

Transition: The FSM stays in S_END indefinitely until the system is reset.